Documentação do Trabalho 1 - Máquina de Estados

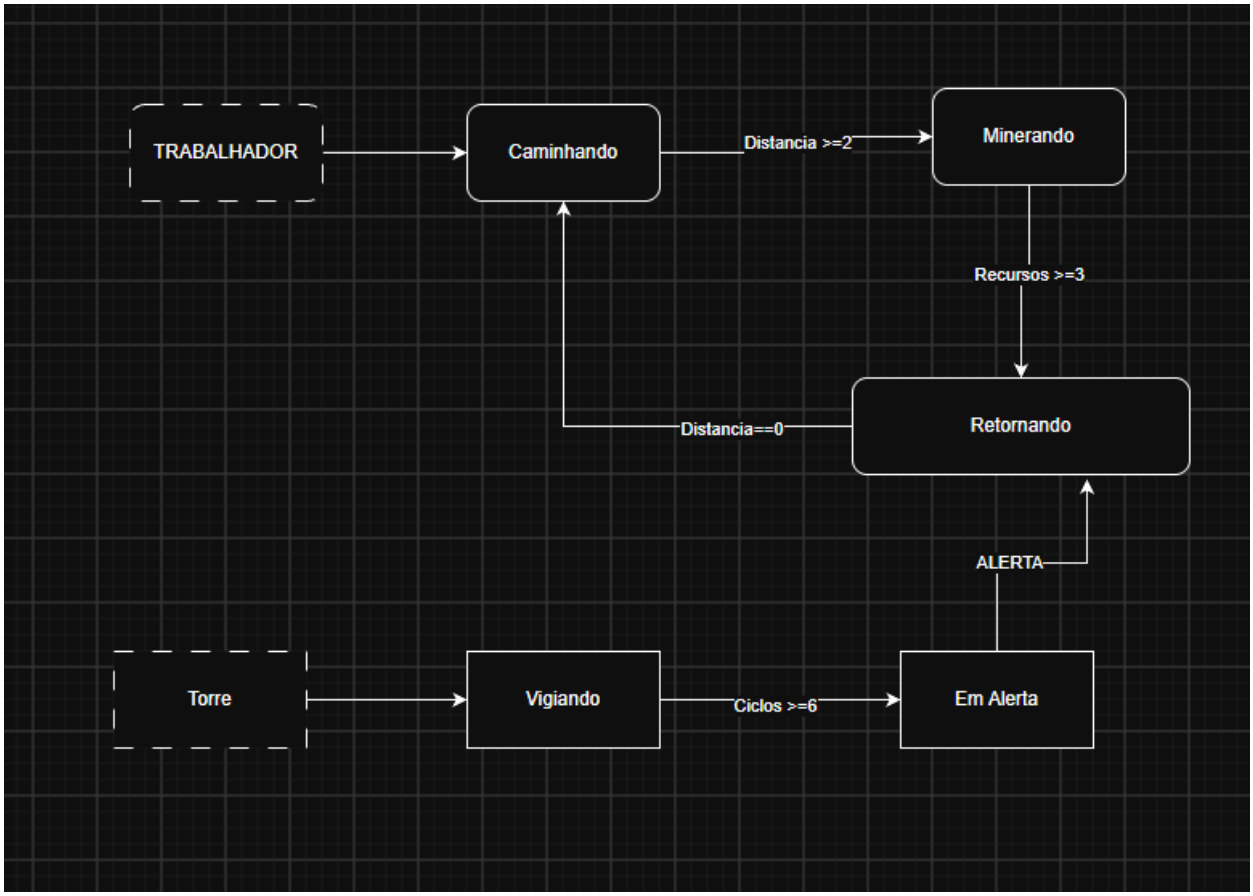
1. Visão Geral do Cenário

Na simulação, há uma base aliada onde um trabalhador realiza sua rotina saindo para coletar recursos em uma mina. Simultaneamente, a base é vigiada por uma torre de defesa. O cenário foca tanto no comportamento rotineiro de coleta quanto na capacidade de resposta do sistema quando uma ameaça se aproxima, demonstrando a comunicação reativa entre as entidades do jogo.

2. Visão Geral de Cada Agente

- Agente A (Trabalhador): É o agente focado na economia. Ele possui três estados básicos que ditam o ciclo de coleta: vai até a mina, minera os recursos e retorna à base. Seu comportamento é guiado por contadores internos de distância e capacidade de carga (recursos).
- Agente B (Torre de Defesa): É o agente de segurança. Ele opera de forma mais passiva, em um estado de vigilância contínua até que um limite de tempo seja atingido, simulando a chegada de uma onda de inimigos, o que a força a entrar em modo de alerta.

3. Diagramas de Estados



4. Tabela de Regras por Agente

Agente A: Trabalhador

Estado Atual	**Ação de Entrada**	**Ação de Execução**	**Ação de Saída**	**Transição / Regra**
Caminhando	Inicia caminhada	Incrementa `distancia`	Chega na mina	Se `distancia >= 2`, vai para Minerando
Minerando	Pega a picareta	Incrementa `recursos`	Mochila cheia	Se `recursos >= 3`, vai para Retornando

Estado Atual	Ação de Entrada	Ação de Execução	Ação de Saída	Transição / Regra
Retornando	Inicia volta à base	Decrementa `distancia`	Guarda recursos	Se `distancia == 0`, vai para Caminhando

Agente B: Torre de Defesa

Estado Atual	Ação de Entrada	Ação de Execução	Ação de Saída	Transição / Regra
Vigiando	Ativa sistemas	Incrementa `ciclosVigia`	Detecta inimigo	Se `ciclosVigia >= 6`, vai para EmAlerta
EmAlerta	Emite Alerta Geral	Atira nos inimigos	Ameaça destruída	Fim da simulação

5. Descrição das Variáveis e Limiares

Variáveis do Trabalhador

- `distancia` (int): Representa o quão longe o agente está da base. Intervalo de 0 a 2. Valor inicial: 0. Limiar para troca de estado: >= 2 (chegou na mina) e == 0 (chegou na base).
- `recursos` (int): Representa a quantidade de minérios na mochila. Intervalo de 0 a 3. Valor inicial: 0. Limiar para troca de estado: >= 3 (mochila cheia).

Variáveis da Torre de Defesa

- `ciclosVigia` (int): Funciona como um timer interno. Valor inicial: 0. Limiar para troca de estado: >= 6 (momento em que a ameaça surge e o estado muda).

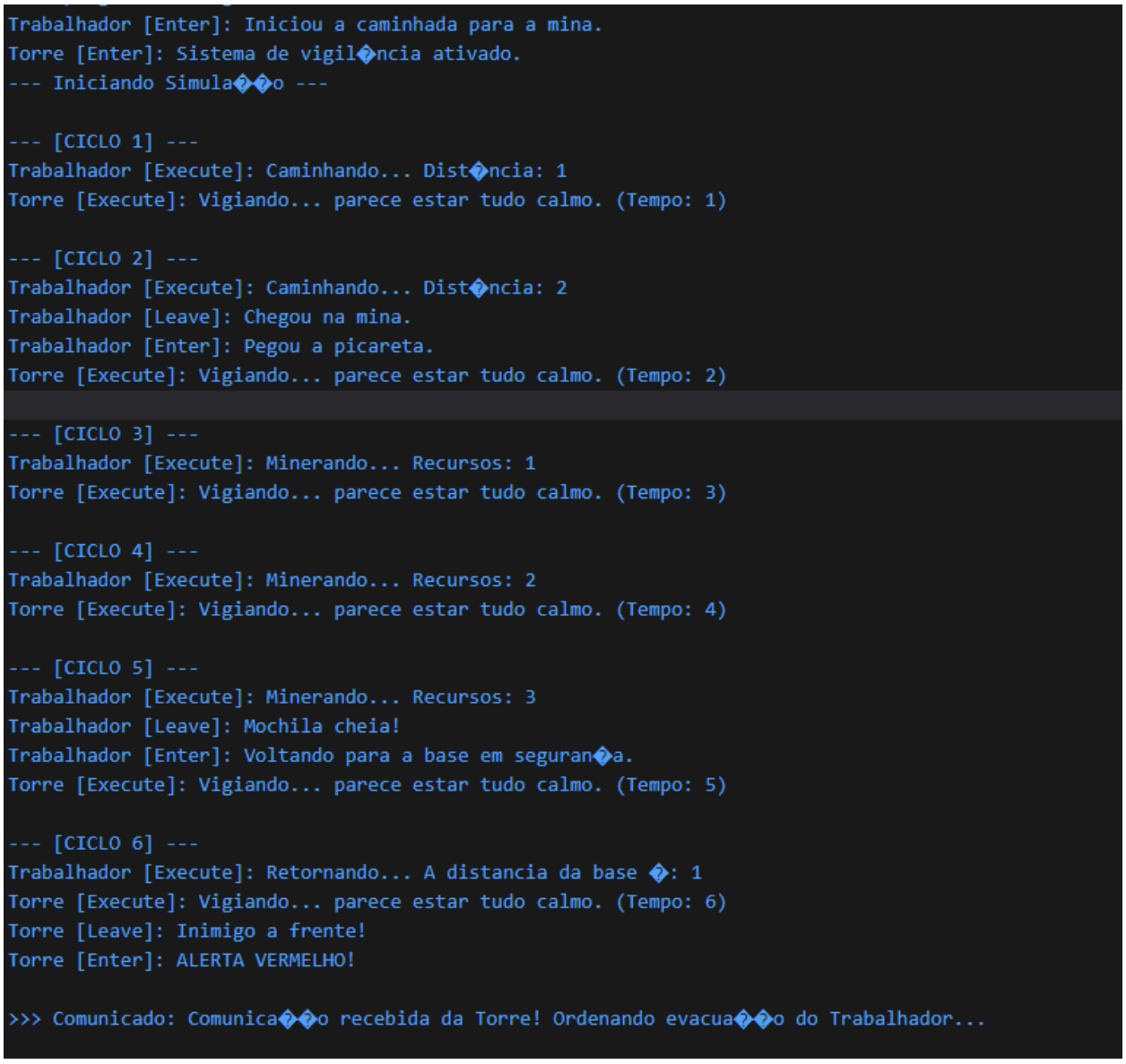
6. Estrutura do Código (Arquivos e Papéis)

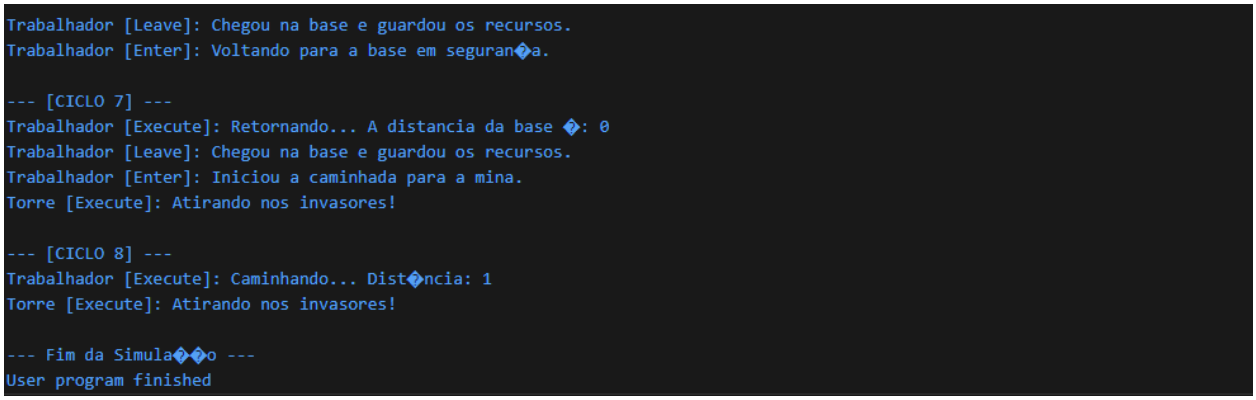
- `States.java`: Interface que define o padrão (métodos enter, execute e leave).
- `Agent.java`: Classe abstrata base que gerencia as variáveis em comum e a lógica de transição (changeState).
- `Trabalhador.java` e `TorreDefesa.java`: Classes que estendem Agent, guardando as variáveis específicas (recursos, distância e timers).

- `[Nome]State.java`: As 5 classes de estados individuais (`CaminhandoState`, `MinerandoState`, e outros.), contendo a lógica e as regras de transição.
- `GestorJogo.java`: O loop responsável por inicializar os agentes, fazer a simulação rodar no console e fazer a parte bônus entre a Torre e o Trabalhador.

7. Resultados Esperados

Ao executar o código, espera-se ver os ciclos e a evolução clara dos estados através de logs no console:





8. Limitações

Como limitação principal, o cenário atual ocorre de forma inteiramente no console via textos, não tem uma movimentação real em malha ou gráficos 2D. Além disso, o Trabalhador foge, mas não toma dano ou interage com a ameaça detectada.

9. Uso de IA

Durante a preparação deste **trabalho prático de Máquina de Estados**, o autor usou o **Gemini (Google)** para **estruturar a base inicial do código em Java, compreender a arquitetura do Padrão State e revisar conceitos de orientação a objetos**. Após usar essa ferramenta, o autor revisou e editou o conteúdo conforme necessário e assume total responsabilidade pelo conteúdo.